

Agile / Scrum Starter Guide

Simple Jira Workflow & SDLC
for the
Pilot Teams Starting Agile / Scrum

August 13, 2026

Make it simple.

Make it easy to understand.

Table of Contents

- [1. The Model at a Glance](#)
- [2. Issue Types \(Just 4\)](#)
- [3. The 5-Status Workflow](#)
- [4. Administration of the Jira Statues](#)
- [5. Jira Board Setup](#)
- [6. Defect Definitions](#)
- [7. Sprint Ceremonies \(The 5 Meetings\)](#)
- [8. The Simple SDLC Cycle](#)
- [9. Development and QA Workflow](#)
- [10. Three Metrics to Start](#)
- [11. Simple Story Point Estimation](#)
- [12. Simple Definition of Sprint Done](#)
- [13. The Scrum Team](#)
- [14. Maturity Roadmap: When to Add What](#)
- [15. A Story is a Shippable Increment of Value](#)
- [16. Common Mistakes New Agile Teams Make](#)
- [17. One-Page Cheat Sheet](#)
- [18. References](#)

Why Start Simple?

The biggest mistake teams make when adopting Agile is over-engineering their process on day one. They create many workflow statuses and automation rules, and complex board configurations — then spend more time managing Jira than building software.

This guide gives you the minimum viable process to run Agile Scrum effectively. Master these basics first, then layer on complexity as your team matures.

1. The Model at a Glance

4 issue types (Epic, Story, Bug, Task)

4 workflow statuses (Open → To Do → In Progress → In Review → Done)

5 ceremonies (Backlog Refinement, Sprint Planning, Daily Standup, Sprint Demo, Retro)

3 metrics (Sprint defect rate, Test coverage, Escape rate)

2-week sprints (the most common starting cadence)

2. Issue Types (Just 4)

Start with only four issue types.

Type	When to Use	Example	Size Rule
Epic	A container holding related stories, typically spanning sprints	Migrate from OAuth 1.0 to OAuth 2.0	Too big for 1 sprint
Story	New feature or user-facing change	"Users can reset their password via email"	Must finish in 1 sprint
Bug	Something is broken	"Login button unresponsive on Safari"	Fix ASAP or next sprint
Task	Less than a story, not user-facing	"Update database connection string"	Must finish in 1 sprint

Writing a Good Story

Use this template:

As a [type of user], I want [something] so that [benefit]. If we want to align to an Aha! template or a template we know will facilitate Claude automation, that's fine too.

 Page: Definitions and templates | Epic Template

Then add 2–5 Acceptance Criteria — simple checkboxes that tell you when it's done:

- ✓ User receives email within 2 minutes of clicking "Reset"
- ✓ Link expires after 24 hours
- ✓ User sees confirmation message after setting new password

When to Add Epics

The business context and use cases are put in the epic. The stories linked to the epic enable them. Once you have stories that belong to the same context or use case (e.g., "User Authentication System"), group them under an Epic.

3. The 5-Status Workflow

This is all we need to start:

Open → To Do → In Progress → In Review → Done

Status	What It Means	Who Owns It	Move Forward When...
Open	Default state when you open a story	Unassigned	It meets the definition of ready (DoR) .
To Do	Ready for Sprint Planning	Unassigned (or pre-assigned)	It has been put into a sprint.
In Progress	Actively being worked on right now	The developer working on it	The work is ready for peer review.
In Review	Any testing happening, Code review, Documentation	The person working on it	When the Sprint definition of done is met.
Done	Sprint done, see the definition in section 8.	N/A	N/A

Rules to Follow

- Limit work in progress: Each person should have max 2 items "In Progress" at a time
- Move items backward if needed: If review finds problems, move back to "In Progress" — that's fine
- Update the board daily: Before or during standup, make sure your board reflects reality
- "Done" means done: Don't move to Done until it meets the definition. The definition of done is outlined later in this document.

What About Documentation?

- Documentation is a lagging activity at this point in time.
- The decision was made by the core working team not to include the Docs role in the scrum team at this time.

4. Administration of the Jira Statues

Status: Open

What it Means: This is the default state of a ticket when it is opened.

Who Owns It: No one is required to own a ticket when it is in Open status. The ticket can be Unassigned.

How It's Used:

- When an Epic and Story ticket type is opened, the layout of the ticket will be aligned to the template, [\[link\]](#).
- While the ticket is in the Open state, it will be refined.
- For a Story ticket, the ticket can't transition from Open to the next status until the definition of ready is met.
- Definition of Ready for a Story
 - Clear user story: The story identifies the actor, desired capability, and intended outcome: As a [user], I want [capability], so that [benefit/outcome]. The benefit is important—it explains the problem being solved and provides context for implementation decisions.
 - Outcome-focused, not over-specified: The story describes what and why, without prescribing every implementation detail. There should be enough clarity to begin work while leaving room for discussion and engineering decisions.
 - Small enough for a sprint: The story can reasonably be completed within a single sprint. Larger stories must be broken down before sprint planning.
 - Prioritized: The story has a clear priority relative to other backlog items, this is a function of the entire Scrum team.
 - Acceptance criteria defined: The story includes clear, testable acceptance criteria that establish what must be true for the story to be considered complete and provide the basis for validation and demonstration.
 - Estimated: The story has been estimated in story points, preferably by the team responsible for delivery or by individuals with sufficient system and domain knowledge. FYI... based on how the Scrum team operates, the story pointing may not be done while in the Sprint Planning ceremony and that's fine.
 - In short: Ready = Clear problem + Clear outcome + Right size + Priority + Acceptance criteria, and potentially even Estimated.

Transition Rules:

- When the DoR is met, the ticket can be transitioned to the To Do status.
- Transitioning to To Do means the ticket is ready for the Sprint Planning ceremony.

Status: To Do

What it Means: A ticket in To Do status means the DoR is met and the ticket is ready for Sprint Planning.

Who Owns It:

- During the Sprint Planning ceremony tickets will be assigned to Developers and Testers.
- It is not good enough to just assign a Developer, you must also assign a Tester. Both Developers and Testers need to plan their work for the sprint.
- When a person is refining a ticket and they have refined it enough to meet the definition of ready, that person can transition the ticket from Open to To Do.

Special Note: During the Sprint Planning ceremony, the entire Scrum team is accountable for making sure the “Needs Documentation” box is filled out correctly. Meaning, for each ticket being played, the entire Scrum team agrees what tickets need documentation and make sure that box is checked ‘yes’ or ‘no’.

How It’s Used:

- The purpose for the To Do status is to identify tickets ready to be included in the Sprint Planning ceremony.
- In Sprint Planning the entire Scrum team selects the work to be performed in the Sprint.
- Only tickets in To Do status should be discussed in the Sprint Planning ceremony.

Transition Rules: When a ticket has a Developer and Tester assigned it can be transitioned to the In Progress status.

Status: In Progress

What it Means: The ticket is in an active sprint and isn’t ready for a code review yet.

Who Owns It: The developer working on it is put as the Assignee and the tester assigned during Sprint Planning is put as the Tester. In the Spring Planning ceremony, the Scrum Master will transition the tickets from To Do to In Progress.

How it is Used:

- The purpose of In Progress is to signify development work is actively underway. In Progress will not be used to signify testing is underway, only development work.
- Development work could be design development, documentation development, code development. Testing is not development, it is testing.
- Sprint Planning should happen on the first day of the sprint.
- When the sprint has been loaded and the entire scrum team agrees what’s in the sprint, the Scrum Master will change the tickets to In Progress and start the sprint.
- In addition, the ticket can be transitioned back to In Progress if it didn’t pass the In Review status check or if a defect was found later in the SDLC and work needs to be done.

Transition Rules:

- When the ticket is ready for a review check it can be transitioned to the In Review status.
- There may be specific required fields on the ticket for a ticket to move from In Progress to In Review, those are mentioned here because they haven't been identified yet.
- The In Review status will be broadly used by developers and testers and potentially the documentation team as well, therefore we are purposefully calling it In Review instead of calling it 'in code review' or other more descriptive names.

Status: In Review

What it Means: The ticket, the work, has progressed to a state where it is ready for peer review or for QA testing.

Who Owns It: Ownership doesn't change. The developer remains the Assignee and the tester remains the Tester. The person doing the development will change the status from In Progress to In Review.

How it is Used:

- The In Review status will be broadly used across all teams that is why we aren't using a more descriptive name. We are trying to limit the number of statuses.
- When a pull request is opened and needs to be reviewed, the ticket should be transitioned to the In Review status. The comments field in the ticket should be actively used so people know the latest progress of the ticket to include people's names as appropriate.
- When a ticket is ready for testing by QA it should be transitioned to the In Review status.
- While QA is testing the ticket should remain in the In Review status. There isn't a separate status for the testing process. Use the comments field in the ticket to document the latest progress of the testing.
- If a bug (defect) is found by QA during testing, a sub-bug will not be opened, the tester should update the comments with the bug details, tag the developer and move the ticket back to the In Progress status.
- When a designer has finished their development and needs it reviewed, they can change the status to In Review and use the comments field to document the latest progress and tag the person they need a review from. When the design review is done and accepted, use the comments field to document the latest progress and transition the ticket back to In Progress and the developer will begin their work.

Special Note: If we don't use sub-bugs it will be very difficult to track 'in sprint defects'. We want to avoid creating separate bug tickets and linking them back to the development ticket. That kind of context switching causes problems. Provocative question. Do we need to care about measuring 'in sprint defects' right now? Is that one of the immediate term goals of the transformation?

Transition Rules:

- When a ticket has met the definition of Sprint done (DoD) it can be transitioned to the Done status.
- There is Sprint done and Release done. Some teams are bound to the monthly release cycle, those teams will have tickets that have all the development and testing work completed and require documentation. The documentation may not be complete yet. In those situations, the Scrum team has completed their work but the increment, the ticket, needs documentation before it's released to customers; therefore the increment is Sprint done but not Release done.
- For Sprints, a ticket can be moved from In Review to Done when the Sprint definition of done has been met. The Sprint definition of done is outlined later in this document.

Status: Done

What it Means: The ticket has met the Sprint definition of done (DoD). Please refer to the transition rules directly above for more context about DoD.

Who Owns It: Ownership doesn't change. The developer remains the Assignee and the tester remains the Tester. The developer is responsible for transitioning the ticket to the Done status.

How it is Used:

- The purpose of the Done status is to signify the increment of work, the ticket, has met all the criteria the R&D organization says needs to be complete before it can leave a sprint.
- If the ticket has met the Sprint definition of done but still requires documentation, the ticket is still moved to the Done status. Documentation is a lagging event. In this situation the ticket isn't Release done but it is Sprint done and no longer worked on by the Scrum team.

Transition Rules:

- There are no transitions from Done.
- Later in the SLDC if QA finds a regression, a new bug ticket is opened and that bug ticket is linked to the development ticket and put in that team's backlog so it can be refined and played.

5. Jira Board Setup

Our First Scrum Board

Create a Scrum board in Jira with these 4 columns:

Column	Maps to Status	WIP Limit
To Do	To Do	No limit (it's your sprint plan)
In Progress	In Progress	2 per person
In Review	In Review	3–5 per role
Done	Done	No limit

Use Simple Quick Filters

Use simple filters making it easy for the Scrum team to see where they are collectively and individually, or point out some obvious hygiene issues.

Tickets with no fix version, No story points, Recently updated, Assigned to me, etc.

As a Scrum team you can decide which filters would be helpful to you.

Open Not Showing on the Scrum Board

The Open status doesn't show up on the Scrum board because those tickets wouldn't have met the definition of ready so they wouldn't have transitioned to To Do and therefore wouldn't have made it to Sprint Planning and therefore wouldn't be playing in a sprint.

Swimlanes (Optional)

If it's helpful to see Bugs and Tasks separate from Stories, use swimlanes by Issue Type. This puts each in a visible row so they don't get lost.

6. Defect Definitions

For Bug tickets, we will begin using Severity alongside Priority. This is a new muscle we will need to build.

Severity	Typical definition (software/SaaS)	Examples
1 – Critical	Complete outage or severe degradation of a production, customer-facing service, affecting all or a large majority of users; no reasonable workaround; major financial, security, or compliance risk.	Entire SaaS product down; authentication down for all users; confirmed security breach; customer data loss.splunk+2
2 – Major	Significant loss of core functionality or major degradation affecting a subset of customers or an internal critical function; business operations impaired but not completely stopped; no acceptable workaround.	Core feature (e.g., checkout, order processing, core workflow) failing for some customers; major performance issues for a significant subset.splunk+2
3 – Moderate	Partial loss or degradation of non-core functionality, or intermittent issues; noticeable but manageable impact; workaround usually available.	Non-critical feature broken; intermittent errors; moderate performance degradation with workarounds.splunk+2
4 – Minor	Low impact issue; minor inconvenience or cosmetic problem; no meaningful impact on business operations.	UI layout issues; minor validation bugs; non-blocking error messages.splunk+2
5 – Trivial / Request	Very low impact defects, cosmetic issues, or general enhancement/feature requests; does not impact current usability.	Typos, cosmetic UI polish, suggestions, non-urgent feature requests.splunk+3

To Do:

- We still need to create a similar chart for our current Priority levels.
- We still need to create a matrix showing the relationship between Severity and Priority.
- We still need to create a matrix showing the service level agreements (SLAs) for a bug ticket resolution based on its Severity and Priority.

7. Sprint Ceremonies (The 5 Meetings)

Scrum has 5 recurring meetings; Backlog Refinement, Sprint Planning, Daily Standup, Sprint Demo, Sprint Retrospective. If the scrum team is in more meetings than this talking about the process, does something need to potentially change?

Backlog Refinement, Prioritization

When: Bi-weekly meeting, but this is ongoing throughout the week for the PM, Tech Lead, and SM

Duration: 1–2 hours max (be thorough to avoid additional one-off meetings)

Who: The entire Scrum team

What you do:

- Reviewing upcoming stories/bugs/tasks to make sure they're ready for sprint planning
- Two sprints worth of items should be refined and estimable
- This is a recurring activity, not a one-time event

Refinement Do's and Don'ts

- Do try and timebox the discussion per item (10 - 15 min max)
- Do ask "can someone pick this up tomorrow and start?" — if their answer is no, it's not refined
- Don't solve the problem in refinement — just clarify the what not the how
- Do ask questions so the entire Scrum team understands the “ask” the reason for the ticket and what it's trying to accomplish
- Don't estimate without the people doing the work in the room
- Do move items that need more research back with a spike attached

Output of a Good Refinement Session

- Top of backlog is prioritized and estimated
- Blocked items flagged with owners to resolve
- PM, Tech Lead, SM are aligned on the backlog prioritization

Prioritization:

- This tells the Scrum team how to prioritize the backlog. Today we use the RICE method.
- The RICE prioritization method scores things using four factors: Reach, Impact, Confidence, and Effort.
- The Formula: $\text{RICE Score} = (\text{Reach} \times \text{Impact} \times \text{Confidence}) / \text{Effort}$
- Scoring Breakdown
 - **Reach** (*Users per timeframe*): How many people will this affect in a given period (e.g., per month)?
 - *Example:* 500 customers/month = **500**

- **Impact** (*Scale of 0.25 to 3*): How much will this improve the outcome for each user?
 - **3** = Massive impact
 - **2** = High impact
 - **1** = Medium impact
 - **0.5** = Low impact
 - **0.25** = Minimal impact
- **Confidence** (*Percentage*): How sure are you about your Reach, Impact, and Effort estimates?
 - **100%** = High confidence (backed by clear data)
 - **80%** = Medium confidence (supported by feedback or benchmarks)
 - **50%** = Low confidence (mostly an educated guess)
- **Effort** (*Person-months*): How much total time will the team need to complete it?
 - *Example*: 1 developer for 2 weeks = **0.5** person-months

Quick Example Comparison, the Higher score gets prioritized over the lower score.

Item	Reach	Impact	Confidence	Effort	Calculation	Final Score	Priority
Quick Checkout Button	1,000	2 (High)	80% (0.8)	0.5 mos	$\$(1,000 \times 2 \times 0.8) / 0.5$	3,200	#1
Dark Mode Theme	2,000	0.5 (Low)	100% (1.0)	1.0 mos	$\$(2,000 \times 0.5 \times 1.0) / 1.0$	1,000	#2

Sprint Planning

When: First day of the sprint

Duration: 2 - 3 hours (be thorough, honest, and transparent)

Who: The entire Scrum team

What you do:

- Pick a sprint goal — one sentence describing what you'll accomplish
- Pull stories from the backlog into the sprint (aim for 80% of what you think you can do)
- Briefly discuss how you'll approach each story
- Everyone agrees: "Yes, we can do this in 2 weeks"

Tip: For your first few sprints, deliberately under-commit. Finishing early feels great and builds confidence. Over-committing and failing feels terrible.

Daily Standup

When: Same time every day

Duration: 15 minutes MAX (set a timer)

Who: Dev team (manager can listen but shouldn't run it)

Each person answers 3 questions:

- What did I finish yesterday?
- What am I working on today?
- Is anything blocking me?

Tip: Actively use the Jira board. Move your cards as you talk. This keeps it focused and updates the board in real-time.

Sprint Demo

When: Last day of the sprint

Duration: 60 - 90 minutes

Who: The entire Scrum team + stakeholders, everyone is welcome if it provides value, but the Scrum team doesn't change the format or process to meet their needs.

What you do:

- Show working software for each completed story (not slides, not code — the real thing)

- Say what you didn't finish and why (this is normal and healthy)
- Collect feedback — write it down as future stories

Sprint Retrospective

When: After the Sprint Review (Demo), but before the next Sprint Planning meeting.

Duration: 60 - 90 minutes

Who: The entire Scrum team (no stakeholders)

Simple format that works:

- What went well? (Keep doing these things)
- What didn't go well? (What frustrated us?)
- What will we try differently next sprint? (Pick just 1–2 changes)
- Did we incorporate previous learnings? Was the outcome achieved?

Tip: The retro is the most important ceremony. It's how you get better. Never skip it.

8. The Simple SDLC Cycle

For a team just starting Agile, your development cycle looks like this:

Write Stories → Plan Sprint → Build & Test → Review, Merge, Regression Testing

Write Stories (Ongoing)

Product writes stories in the backlog with acceptance criteria. The Product Manager, the Tech Lead / Architect, and the Scrum Master continuously review them every week "refinement" and prioritizing the backlog.

Plan Sprint (Day 1)

Sprint Planning, the scrum team picks stories for the next 2 weeks.

Build & Test (Days 1–8)

Where applicable, UX/UI pick up stories and do design work. Developers pick up stories, write code, write/execute unit tests. QA does functional testing alongside development. Defects are raised and resolved before full regression testing. Docs is writing rough drafts alongside QA and Developers.

Review, Merge, Full Regression Testing (Days 3–10)

Team members review each other's code. Keep PRs small so reviews are fast (under 1 hour to review). Tickets are merged to (x) branch for QA weekly full regression testing. Defects are raised and resolved within the sprint.

UAT Certification

Running full regression testing on UAT to certify the release isn't captured in this simplified SDLC. Release certification is being viewed as a separate effort and we have to take into account QA's bandwidth when we get close to a release. The sprint that overlaps with release certification may need to be smaller in scope to allow QA the bandwidth they need to certify the code base in UAT.

Patch Releases

Patch releases are one off's and aren't captured in this simplified SDLC, room will need to be made in a sprint for a patch. A story may need to be moved out of the sprint to make room for the patch.

9. Development and QA Workflow

This traces APP-2330 as a reference example.

Tracing APP-2330

The Feature branch was created and had multiple commits. The PR merged into the RC branch. QA did integration and regression testing on the RC branch. The RC branch was merged into Main. The RC branch was also back-merged into Develop to keep that branch current.

The individual feature branch wasn't deployed anywhere QA could test. The RC branch got deployed to a staging/QA environment.

So the flow was:

1. Dev built & unit tested on a feature branch. Note, it's important to rebase the feature branch with the RC branch prior to unit testing.
2. Code review (peers) happened on the feature branch.
3. Merged the Feature branch to the RC branch.
4. The RC branch was deployed to a test/staging environment for QA.
5. QA tested on the RC branch, (integration + regression)
6. The RC branch was merged to Main

Is this a Concern

- QA doesn't engage until step 5 — after the code has already been merged and deployed to the RC branch.

- In the specific case of this ticket, APP-2330, there was a 14-day gap between the Code Review step and QA being able to start testing.
- This feels more like a sequential handoff than an Agile workflow.

10. Three Metrics to Start

Don't drown in metrics. Track just three things for our first 3–6 months:

Metric 1: In Sprint Defect Rate

What: How many defects (bugs) QA logs per team per sprint.

Why: Helps reduce waste.

How to use it: No more than 1 defect per engineer per sprint.

Special Note: Go to the page that outlines the In Review status. [\[link\]](#) It has a note about sub-bugs that directly impacts this metric. It might be difficult to capture this metric in its totality. If we don't use sub-bugs we could still capture part of this metric. After the sprint is closed, later in the SDLC if a regression bug is found a new bug ticket is opened and linked back to the development ticket. We would be able to capture and measure that.

Metric 2: Test Coverage

What: The percentage of your code that gets executed when your automated tests run. Top Line Metric: 80% code coverage, if you have 100 lines of code and your tests touch 80 of them, you have 80% coverage.

Formula: Code Coverage % = (Lines Executed by Tests ÷ Total Lines of Code) × 100.

Why It Helps: Tells you how much of your code is "protected", if someone breaks a tested line, a test will fail and catch it before it ships. It acts as a safety net. You can change code with confidence if tests cover it. It identifies dead zones, code that has zero tests is where bugs can hide.

How to Use It: 80% is the industry standard target for most SaaS teams. Companies such as Google, Microsoft, and Atlassian all reference this threshold. Don't aim for 100%, you hit diminishing returns past 80-90% wastes time. If possible, enforce it in CI: fail the build if a PR drops coverage below 80%.

Metric 3: Escape Rate, Custom Reported Defects

What: The percentage of defects (bugs) that slip past your testing and get discovered in production by customers rather than being caught internally before release.

Formula: The generic benchmark is ≤ 1 customer reported bug per engineer per month. The benchmark is derived from Capers Jones' research across 12,000+ software projects (Applied Software Measurement, 2008; The Economics of Software Quality, 2011). Jones established that average US software achieves ~85% Defect Removal Efficiency, with high-performing organizations at 90-95%.

Why It Helps: It measures how effective your testing process actually is, are you catching bugs before customers see them? A rising escape rate is an early warning that your testing has gaps. A falling escape rate means your team is getting better at quality which directly correlates to customer trust and support ticket volume.

How to Use It: Track the number of customer reported defects per month against the number of engineers shipping code. Take the total number of customer reported bugs linked back to the scrum team and divide that by the number of Engineers shipping code in that scrum team. If the percentage is less than or equal to 1 you have met the benchmark. If the percentage is greater than 1, you haven't met the benchmark.

Metrics to Add Later

- Velocity -- The total number of story points your team completes (moves to Done) in a single sprint
- Cycle Time -- how long stories take from start to finish (aim for shorter)

11. Simple Story Point Estimation

Story points measure complexity, not time for development work. Use this simple scale:

Points	Complexity	Example
1	Trivial — you could do it in your sleep	Fix a typo, change a color
2	Simple — straightforward, no unknowns	Add a new field to a form
3	Small — a little thinking required	Create a new API endpoint (simple)
5	Medium — some complexity, maybe a new pattern	Build a password reset flow
8	Large — significant work, multiple pieces	Add user roles and permissions
13	Very large — consider splitting this	Build a complete notification system

Estimation Tips

- If it's bigger than 8 points, try to split it into smaller stories
- If there's disagreement, discuss briefly, then re-vote

- Estimation improves with practice
- Don't convert points to hours. They're intentionally abstract.

Capturing Story Points

- Jira reporting and KPIs use the default Story Points field. We (SnapLogic) have created custom story point fields for QA and Docs to capture their effort.
- This story pointing model can be used by all teams but the default Story Points field is for Dev estimates.
- We will use the existing QA custom Story Points field and add it to the story, bug, and task, ticket types.

12. Simple Definition of Sprint Done

A story is "Sprint Done" when ALL of these are true:

- ☐ Code is written, unit testing is complete (the minimum code coverage by unit tests is 80%)
- ☐ Feature branch testing (no P0 / P1 defects) / Functional testing is complete
- ☐ Any defects QA found have been resolved
- ☐ The acceptance criteria from the story are all met
- ☐ The sprint demo is complete

That's it. 5 checkboxes. If any one of them isn't true, the story stays in "In Progress" or "In Review." Don't move it to Done just because the sprint is ending — incomplete work moves to the next sprint. No part 2 stories, no workflow manipulation. Simple. Straight forward. Easy to understand and execute.

13. The Scrum Team

Product Manager (PM)

Owns discovery and prioritizes the backlog, defines acceptance criteria, and represents business stakeholders. Sets the work the team will execute against.

Scrum Master (SM)

Keeps the team's process running smoothly — facilitating ceremonies, removing blockers, and protecting the team from distractions so they can focus on building.

Developers

Tech Lead / Architect

Owns technical direction, coding standards, and architecture decisions, e.g., choosing technologies, designing how systems connect, and ensuring the code stays maintainable and scalable as the product grows.

UI/UX Designer (UI/UX)

Designs how the product looks and how users interact with it, e.g., creating wireframes, mockups, and user flows so the team builds something intuitive before writing code.

Software Engineers

Write the code that turns stories into working software, e.g., building features, fixing bugs, writing tests, and reviewing each other's work through pull requests.

Quality Assurance (QA)

Tests the software to make sure it works correctly before customers see it, e.g., finding bugs, verifying acceptance criteria, and confirming nothing else broke along the way.

14. Maturity Roadmap: When to Add What

Don't try to implement everything at once. Here's a suggested timeline based on when teams typically feel the need for each practice:

Timeframe	Add This	Why Now
Month 1–2 (Getting Started)	• 4-status workflow • 4 issue types • 5 ceremonies • 2-week sprints • Story points	Learn the basics without overwhelm
Month 3–4 (Finding Rhythm)	• "Blocked" status • Velocity tracking • Basic code review process	You now have enough stories to need organization; you've hit your first blockers
Month 5–6 (Improving Flow)	• WIP limits • Cycle time tracking • Automated tests in CI • Quick filters on board	You're doing the basics; now optimize how fast work flows through
Month 7–9 (Scaling Up)	• 6–8 status workflow • Deployment automation • Sprint commitment tracking • Bug severity classification	Team is confident; process can handle more structure
Month 10–12 (Maturing)	• DORA metrics • Jira automations • Feature flags • Release management • Cross-team dependencies	You're now an intermediate Agile team with real data to drive decisions

15. A Story is a Shippable Increment of Value

Developers and QA will use the same ticket.

The Scrum Guide is clear: a Product Backlog Item (User Story) isn't "Done" until all work — including testing — is complete. QA doesn't get a separate, standalone ticket. The open debate is whether to include Docs as well.

QA will use the same ticket as Development

QA works directly on that ticket. This is the purest Scrum approach. QA can update the comments field similar to the Developers. Anyone looking at the ticket and reading the comments will know what state the ticket is in. Using subtasks is acceptable if we want to try that again.

What we will not do

We will not create a completely separate QA ticket and link it to the Development ticket. This fragments the work and breaks the Scrum principle that a ticket is a shippable increment of value.

Reference

The Scrum Guide (2020) — <https://scrumguides.org/scrum-guide.html>

"Work cannot be considered part of an Increment unless it meets the Definition of Done."

The guide makes no mention of separate QA tickets. It treats "Done" as a holistic state — the Developers (which includes QA in Scrum) are collectively accountable for delivering a complete, tested increment.

16. Common Mistakes New Agile Teams Make

Over-planning the sprint: Don't try to fill every hour. Leave 20–30% buffer for surprises, bugs, and meetings. If you finish early, pull from the backlog.

Treating the standup as a status report: It's not a meeting for the manager. It's for the team to sync with each other. Keep it peer-to-peer, 15 minutes, don't drain topics.

Moving things to Done that aren't done: Pressure to "look good" at sprint end leads to marking things Done prematurely. Incomplete work moved to the next sprint is fine, misrepresenting Done is not.

Skipping the retrospective: "We're too busy." This is exactly why you need the retro — to figure out why you're too busy. It's 30 minutes every 2 weeks. Protect it.

Not limiting work in progress: Starting 5 stories and finishing 0 is worse than starting 2 and finishing 2. Rule of thumb: Never have more items In Progress than team members.

Making sprints longer because you can't finish work: The fix is smaller stories or less commitment — not longer sprints. 2-week sprints give you fast feedback on whether your planning is working.

Spending too much time on Jira instead of building: If updating Jira takes more than 5 minutes per day per person, your process is too complex. Simplify until it's lightweight.

17. One-Page Cheat Sheet

Our Sprint Calendar (2-Week Sprint)

Day	Event	Duration
First Day of the Sprint	Open the sprint, Sprint Planning	2-3 hours
Every day	Daily Standup	15 min
Last Day of the Sprint	Sprint Review (Demo)	60–90 min
Last Day of the Sprint	Sprint Retrospective, Close the sprint	60–90 min
First Day of the Next Sprint	Next Sprint Planning begins	—

Our Board

| To Do | In Progress | In Review | Done |

Our Definition of Sprint Done

A story is "Sprint Done" when ALL of these are true:

- ☐ Code is written, unit testing is complete
- ☐ Feature branch testing is complete
- ☐ Any defects QA found have been resolved
- ☐ The acceptance criteria from the story are all met
- ☐ You could demo it to someone and it would work

Our Story Point Estimation Scale

1 = trivial | 2 = simple | 3 = small | 5 = medium | 8 = large | 13 = split it

Prioritize Using RICE

RICE Score = (Reach x Impact x Confidence) / Effort

18. References

This guide was developed by taking information from the following sources:

Reference 1: Atlassian — Agile Coach (Beginner Track)

URL: <https://www.atlassian.com/agile/scrum>

Atlassian's free Agile Coach resource covers Scrum basics with short articles and diagrams. Start with "What is Scrum?" and "Sprint Planning." Explains each ceremony in plain language with practical tips for new teams. Also covers how to set up your first Jira Scrum board.

Reference 2: Scrum.org — The Scrum Guide (Official)

URL: <https://scrumguides.org/scrum-guide.html>

The official Scrum Guide by Ken Schwaber and Jeff Sutherland (the creators of Scrum). Only 13 pages long — intentionally brief. Defines the roles, events, and artifacts that make up Scrum. Read this once to understand the "why" behind each element. It's the authoritative source when your team disagrees about how Scrum is "supposed" to work.

Reference 3: Atlassian — Jira Workflows Overview

URL: <https://www.atlassian.com/software/jira/guides/workflows/overview>

Atlassian's official documentation on Jira workflows — what statuses, transitions, and resolutions are, how to configure them, and how they map to your board columns. Includes a visual workflow editor walkthrough. Read this when you're ready to customize beyond the default Jira workflow.

Reference 4: Mountain Goat Software — Introduction to Scrum (Video)

URL: <https://www.mountaingoatsoftware.com/agile/scrum>

Mike Cohn's site (author of "Agile Estimating and Planning") provides a beginner-friendly overview of Scrum with practical advice on user stories, estimation, and sprint planning. Particularly good for understanding story points and why they work better than estimating in hours.